CDFD Runtime Paper 04: Engine, CLI, and Reproducible Execution

Steve Bico Mujjabi, MD
Independent Researcher
ORCID: 0009-0001-0556-5516

May 2026

Abstract

Executive synthesis. The CDFD Runtime is the executable layer of the CDFD program. This paper documents the current architecture: state arrays in `engine/state.py`, physics and domain updates under `engine/`, domain adapters under `domains/`, CDFL under `dsl/`, shared command logic under `runtime/runner.py`, and the CLI entrypoint `cdfd.py`. The CLI is the first serious interface because it supports reproducible simulations, paper diagnostics, validation, export, and automation before the web application is expanded.

Greek-symbol convention. Unless a manuscript states a tighter equation-local meaning, Greek symbols are local CDFD variables or coefficients, not universal constants. Here Φ denotes driving flux, Ψ_s denotes the adaptive operating ratio, Ω denotes a domain or state space, and Λ denotes the Life Number where used. The coefficients α, β, and γ denote accumulation or reinforcement, relaxation or decay, and diffusion, coupling, or redistribution. The symbols δ and ϵ denote perturbation or tolerance scales, while η, λ, μ, ν, ρ, σ, τ, and ϕ denote local efficiency, rate, baseline, frequency, density or correlation, noise or variance, time-scale, and phase or field terms. Subscripts restrict any coefficient to the named pathway, tissue, domain, or experiment.

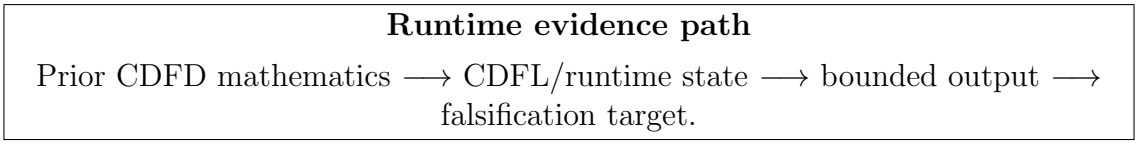

Figure 1: Conceptual schematic for the runtime papers. The engine translates the mathematical frame from the prior CDFD papers into executable CDFL/runtime diagnostics; candidate outputs remain tied to explicit tests before any stronger claim is made.

1 Prior CDFD Basis

This manuscript does not rederive the mathematical core of CDFD. It uses the Mujjabi Adaptive Operating Ratio and the constrained-vacuum notation introduced in Part I [1], the torus-knot hierarchy and boundary-mode work from the Part I sequence [2, 3], the

public balance law developed in the Part I capstone [4], the Part I transport tests [5], and the Life Number and tri-regime biology bridge from Part II [6]. The runtime papers therefore act as an execution layer: they keep the mathematics connected to commands, outputs, falsification tests, and domain-specific limits.

2 Architecture Principle

The core engine must not depend on the web app. The CLI and web app call the same backend, and both preserve a strict separation between:

- numerical state evolution;
- domain interpretation;
- command presentation;
- output serialization;
- validation and finite-value checks.

3 State and Kernel

The state module stores Φ, C, S, M_s, Ψ_s, and domain-specific metadata. The kernel module orchestrates updates and isolates domain failures so one module does not halt an entire run.

The kernel is deliberately conservative. It returns structured state, active constraints, status flags, and metadata rather than hidden side effects. Every domain module is optional; the physics step remains the common substrate.

4 Shared Runtime Utilities

The upgraded runtime exposes `runtime/diagnostics.py`. This module centralizes tools that were previously scattered through release scripts:

- strict JSON cleaning and non-finite audits;
- scalar and array forms of Ψ_s;
- the Life Number Λ;
- bounded adaptive updates for toy diagnostics;
- Part II aromatic source-mix rows and photochemical endpoint guardrails;
- state summaries and result envelopes with provenance.

5 CLI-First Interface

The CLI entrypoint is `cdfd.py`. From the runtime root:

```
python cdfd.py info
python cdfd.py domains
python cdfd.py diagnostics
python cdfd.py demo physics --steps 4 --nx 8 --ny 8 --json
python cdfd.py demo origins_of_life \
  --source-scenario mixed_source_surface_trap
python cdfd.py validate examples/heat_flow.cdfl
python cdfd.py run examples/heat_flow.cdfl \
  --out outputs/heat_flow_run.json
python cdfd.py export outputs/heat_flow_run.json \
  --out outputs/heat_flow_export.json
python cdfd.py auth --api-key-file app_key.txt --json
```

The CLI calls `runtime/runner.py`. Domain demos return per-step traces and adapter-specific `domain_diagnostics` when available. The optional Streamlit layer (`python -m webapp.run_server`) reads the same backend; `webapp/neo4j_ontology.py` is kept separate from the `ontology/` package to avoid import shadowing.

6 Application Credentials

The final CLI surface includes an app-auth boundary. A calling product can pass an API key directly, through a key file, or through an environment variable. The runtime never stores the raw key in output. It records only whether the key matched the configured allowlist and, when a key is supplied, a short redacted fingerprint for audit. This gives VOS and future apps a practical gate while keeping LLM credentials and user-workflow policy out of the engine.

7 Reproducible Output Contract

Every serious command returns:

- `status`: whether the command succeeded;
- `kind`: result type;
- `provenance`: runtime name, CDFL language tag, timestamp, command, Python version, and platform;
- `finite_audit`: paths to non-finite values if any;
- `payload`: the domain, CDFL, or export result.

This is essential for paper diagnostics. A figure or discovery claim without parameters, command, and output path is not part of the canonical runtime evidence chain.

8 Experiment Integration

The experiments directory is not a separate mythology layer. It is the stress test surface for the runtime:

- `experiments/outputs/discovery_vacuum_hysteresis.h5`;
- `experiments/outputs/discovery_knot_n7.h5`;
- `experiments/outputs/discovery_ool_phase_results.h5`;
- `experiments/outputs/frontier_sweep.h5`;
- `experiments/outputs/gigamarathon_1000_results.h5`;
- `experiments/outputs/part_ii_runtime_diagnostics.json` (regenerated by `python cdfd.py diagnostics`).

The report `experiments/reports/FINAL_RELEASE_EXPERIMENT_SELECTION.md` selects which outputs are suitable for public mention and how cautiously they are framed.

9 Current Verification

The CLI/runtime upgrade pass was checked with:

```
python -m pytest -q tests/test_cli_runtime.py \
  tests/test_new_engine_upgrades.py
```

The recorded result was 15 passing runtime and native-engine tests. This is not full validation of the CDFD theory; it is evidence that the command backend, CDFL example, Part II diagnostics, domain traces, and selected native engine upgrades execute coherently.

10 Conclusion

The CDFD Runtime is becoming a reproducible scientific instrument rather than only a manuscript companion. The CLI is the correct first interface because it supports exact commands, saved outputs, test automation, and falsifiable diagnostics before visual presentation is layered on top.

References

[1] Steve Bico Mujjabi, MD. *Vortex Stability in a Constrained Transport Vacuum and the Origin of the Fine-Structure Constant.* CDFD Part I Paper I, preprint, 2026.

[2] Steve Bico Mujjabi, MD. *The Universal Torus Knot Hierarchy: Z_n Symmetry, the Cinquefoil Family, and the General Structure of CDFT Particle Families.* CDFD Part I Paper VI, preprint, 2026.

[3] Steve Bico Mujjabi, MD. *Even-n Torus Knots in CDFT: The Exclusion of $T(2,2)$, the Extension to $T(2,2m)$, and the Anti-Phase Mass Scaling Law.* CDFD Part I Paper IX, preprint, 2026.

[4] Steve Bico Mujjabi, MD. *The Vacuum Equation of State: Deriving Torus Knot Self-Consistency from the Public CDFD Balance Law* $\Psi = \Phi/C$. CDFD Part I Paper X, preprint, 2026.

[5] Steve Bico Mujjabi, MD. *Friction, Superconductivity, Plasma States, and Transport Mysteries: Observable Tests of Adaptive Constraint*. CDFD Part I Paper XII, preprint, 2026.

[6] Steve Bico Mujjabi, MD. *CDFD Part II: Origins of Life and Tri-Regime Bioenergetics*. Public release archive, 2026, `https://doi.org/10.5281/zenodo.20264779`.